

Job Market Analytics Platform

Technical Implementation and Architecture

Author

June 21, 2026

Contents

1	Core Platform & Infrastructure Topology	7
1.1	Overview	7
1.2	Core Platform Topology	9
1.3	Next.js Production Containerization	10
1.4	Database Health Probing & Startup Sequencing	11
1.5	Internal Docker DNS Service Discovery	12
1.6	Environment Variable Strategy	12
1.7	Elasticsearch Configuration	13
1.8	Technology Choice: Docker Compose vs Kubernetes	13
2	Data Acquisition Pipeline (Web Scraping)	14
2.1	Overview	14
2.2	Scraping Architecture and CI/CD Integration	14
2.2.1	CI/CD Dual-Runtime Pipeline	15
2.2.2	Technical Flow	15
2.3	Playwright vs Traditional Scrapers	16
2.3.1	Technology Choice: Playwright vs Alternatives	16
2.3.2	Anti-Scraping Countermeasures	17
2.4	Concurrency Control and Data Freshness	18
2.5	Data Normalization Pipeline	19
2.5.1	Phase 1: Pattern-Based Cleansing	19
2.5.2	Phase 2: Semantic Normalization and Feature Extraction	19
2.5.3	Phase 3: Deterministic Content Hashing	20
2.5.4	Phase 4: Persistence	20
2.5.5	Normalization Flow Diagram	20
2.6	Key Quantitative Metrics	21

3	Polyglot Persistence and Search Infrastructure	22
3.1	Overview	22
3.2	Polyglot Persistence Justification	22
3.2.1	Why Not a Monolithic Database?	23
3.3	Supabase (Core Relational Database)	23
3.3.1	Structured Entities	23
3.3.2	Dual-Client Architecture	26
3.4	Redis (Session, Cache, and Security Layer)	26
3.4.1	Singleton Connection Pattern	26
3.4.2	Redis Key Space Architecture	26
3.4.3	Core Workloads	27
3.5	Elasticsearch (Full-Text Search Engine)	27
3.5.1	CQRS Data Flow	27
3.5.2	Data Synchronization	27
3.5.3	Advanced Mapping and Indexing	28
3.6	Technology Choice: Elasticsearch vs Alternatives	28
3.7	Key Quantitative Metrics	29
4	Full-Text Search Engine (Elasticsearch)	30
4.1	Search Architecture & Rationale	30
4.2	Index Schema & Mapping Strategies	31
4.3	Data Ingestion & Deduplication Pipeline (Sync Flow)	32
4.4	Vietnamese NLP & Heuristic Normalization Logic	33
4.5	Multi-Field Search & Relevance Tuning	34
4.6	Aggregation & Memory-Resident Enum Caching	36
4.7	Key Quantitative Metrics	36
5	Authentication & Authorization	37
5.1	Architecture Overview	37
5.1.1	Technology Choice: Supabase Auth vs Alternatives	37
5.2	Authentication Flow	38
5.3	Route Protection Strategy	39
5.4	Advanced Security: JWT Blacklisting via Redis	40
5.4.1	Optimized Storage Process:	40
5.4.2	Fail-Closed vs Fail-Open: Deliberate Security Trade-offs	41
5.5	Rate Limiting	41
5.5.1	Mechanism:	41
5.5.2	Design Decisions:	42

5.6	Form Management and Input Validation	42
5.7	Key Quantitative Metrics	42
6	Core Frontend Layer (Next.js)	44
6.1	Overview	44
6.1.1	Technology Choice: Next.js App Router vs Alternatives	44
6.2	SSR Data Fetching and User-Aware Rendering	45
6.3	Faceted Search Architecture	45
6.3.1	Hybrid State Management Strategy	46
6.3.2	Supported Filter Parameters	47
6.4	Client-Side Vietnamese NLP Processing	47
6.4.1	Location and Geographic Parsing (<code>splitLocations</code>) .	47
6.4.2	Unstructured Salary Categorization (<code>getSalaryBuckets</code>)	48
6.4.3	Experience Tier Mapping (<code>getExpBuckets</code>)	48
6.5	Page Architecture	49
6.6	Key Quantitative Metrics	49

Chapter 1

Core Platform & Infrastructure Topology

1.1 Overview

The foundation of the CareerIntel platform is a highly resilient, polyglot microservices architecture. Designed to integrate web scraping, full-text search, and real-time user interfaces, the infrastructure relies on Docker Compose to orchestrate seven concurrent containers. This topology ensures that the Next.js frontend, the Elasticsearch search engine, and the core PostgreSQL database operate seamlessly within a secure, internal software-defined network.

This chapter focuses on the four core platform services that underpin the data management and user-facing layers (Next.js, Elasticsearch, Redis, MongoDB). The remaining three services `chatbot-api`, `celery-worker`, and `qdrant` form the AI Chatbot subsystem and are documented in the companion thesis (Thesis 1).

The following system-wide data flow diagram illustrates how each chapter in this thesis corresponds to distinct architectural stages and data movements across the platform:

```
flowchart TD
    subgraph Ch2["Chapter 2: Data Acquisition"]
        SCRAPE["Playwright Scrapers"]
        NORM["Offline Normalization"]
    end
    end
```

```

subgraph Ch3["Chapter 3: Polyglot Persistence"]
    SUPA[("Supabase (PostgreSQL)<br/>Source of Truth")]
    REDIS[("Redis Cache<br/>Sessions & Security")]
end

subgraph Ch4["Chapter 4: Search Engine"]
    ES[("Elasticsearch Index")]
    SYNC["sync.ts Sync Pipeline"]
end

subgraph Ch5["Chapter 5: Auth & Security"]
    AUTH["Supabase Auth / Server Actions"]
    SEC["JWT Blacklist & Rate Limiting"]
end

subgraph Ch6["Chapter 6: Next.js Frontend"]
    UI["App Router UI / Pages"]
    STATE["URL-driven Search State"]
end

%% Flows
SCRAPE -->|"Upsert raw"| SUPA
NORM -->|"Upsert normalized"| SUPA
SUPA -->|"Batch pagination"| SYNC
SYNC -->|"Bulk push"| ES

UI -->|"1. Auth request"| AUTH
AUTH -->|"2. Identity check"| SUPA
AUTH -. ->|"3. Blacklist/Rate limit"| SEC
SEC -. ->|"Read/Write"| REDIS

UI -->|"4. Faceted Search"| STATE
STATE -->|"5. Search query"| ES

```


1.2 Core Platform Topology

The primary user-facing and data management layers of the application are supported by the following containerized services:

- **next-app**: The Next.js 16 (App Router) frontend serving both Server-Side Rendered (SSR) web pages and API routes (port 3000). It acts as the primary gateway for users interacting with the job search interface and user profiles.
- **elasticsearch**: An Elasticsearch 8.13.0 node optimized for complex full-text queries and faceted filtering, essential for querying thousands of normalized job postings.
- **redis**: Operates as an in-memory session cache and handles rapid enumerations (e.g., job tracking state), utilizing the **ioredis** library on the Next.js side. The container is configured with **password-based authentication** (`-requirepass`) to prevent unauthorized access, and employs **Append-Only File (AOF) persistence** (`-appendonly yes`) to ensure that cached data survives container restarts by writing every mutation to an append-only log on disk.
- **mongodb**: Provides document-based storage for flexible data schemas, utilized heavily by the Next.js API routes for managing user chat sessions via the `chatService`.

(Note: The core user and job data are persisted in a managed Supabase PostgreSQL instance, which the containerized services access securely via Service Role Keys).

graph TB

```
subgraph "Host Machine"
    OLLAMA["Ollama (Host GPU)<br/>:11434"]
end

subgraph "Docker Compose Network (7 Containers)"
    subgraph "Core Platform (Thesis 2)"
        NEXT["next-app<br/>Next.js 16<br/>:3000"]
        ES["elasticsearch<br/>ES 8.13.0<br/>:9200, :9300"]
        REDIS["redis<br/>Alpine + AOF + Auth<br/>:6379"]
    end
end
```

```

        MONGO["mongodb<br/>Mongo Latest<br/>:27017"]
    end

    subgraph "AI Subsystem (Thesis 1)"
        CHATBOT["chatbot-api<br/>FastAPI<br/>:8000"]
        CELERY["celery-worker<br/>Celery ML Queue"]
        QDRANT["qdrant<br/>Vector DB<br/>:6333"]
    end

    SHARED["shared_tmp<br/>(Volume)"]
end

NEXT -->|"depends_on<br/>service_healthy"| REDIS
NEXT -->|"depends_on<br/>service_healthy"| MONGO
NEXT -->|"depends_on<br/>service_healthy"| ES
CHATBOT -->|"depends_on<br/>service_healthy"| REDIS
CHATBOT -->|"depends_on<br/>service_healthy"| ES
CHATBOT -->|"depends_on<br/>service_started"| QDRANT
CELERY -->|"depends_on"| REDIS
CELERY -->|"depends_on"| QDRANT

CHATBOT <--> SHARED
CELERY <--> SHARED
CHATBOT -->|"host.docker.internal"| OLLAMA
CELERY -->|"host.docker.internal"| OLLAMA

```

1.3 Next.js Production Containerization

The `next-app` service is deployed using a highly optimized, multi-stage Dockerfile (38 lines) designed specifically for Node.js environments.

1. **Stage 1 (deps)**: Utilizes `node:20-alpine` and `npm ci` to cleanly install dependencies, maximizing cache hits during the build process.
2. **Stage 2 (builder)**: Compiles the React application. Telemetry is explicitly disabled (`NEXT_TELEMETRY_DISABLED=1`) to prevent external analytics requests during CI/CD. The build outputs an optimized `.next` standalone directory.

3. **Stage 3 (runner)**: The final production image. It explicitly copies only the five necessary compilation artifacts (`package.json`, `node_modules`, `public`, `.next`, `next.config.js`) from the builder stage. This drastically reduces the final attack surface and image size, establishing a secure environment configured tightly around `NODE_ENV=production`.

1.4 Database Health Probing & Startup Sequencing

In a distributed microservices environment, strict boot sequencing is mandatory to prevent cascade failures when upstream dependencies are unresponsive. CareerIntel implements comprehensive database health probing directly within the `docker-compose.yml`.

Most application services do not blindly depend on container startup; instead, they utilize the `depends_on` configuration coupled with `condition: service_healthy` to gate boot sequence.

Service	Health Check Command	Interval	Start Period	
Redis	<code>redis-cli -a \$REDIS_PASSWORD</code>	10s	10s	
MongoDB	<code>mongosh -eval "db.adminCommand('ping')"</code>	15s	20s	
Elasticsearch	<code>curl _cluster/health (status: green/yellow)</code>	30s	60s	

The Next.js and FastAPI application containers are instructed by Docker Compose to halt their startup procedures until these specific health conditions evaluate to true, guaranteeing a robust boot process. Notably, the Redis health check includes the authentication password flag (`-a`), verifying not just TCP connectivity but the full authentication layer. MongoDB's generous 20-second start period accommodates disk initialization latency, while Elasticsearch's 60-second start period allows time for JVM warmup and shard allocation.

However, two exceptions are intentionally designed in this sequencing:

1. **Celery Worker**: The `celery-worker` service utilizes a basic `depends_on` array containing `[redis, qdrant]` without health conditions. This allows the worker to launch concurrently with database start, as Celery natively handles connection retries for its message broker and lazily initializes its connection to the Qdrant client only when a CV processing task is dequeued.

2. **Chatbot API MongoDB Connection:** The `chatbot-api` service connects to MongoDB at runtime for session history logs, but it does not declare a `depends_on` health condition for the `mongodb` service. Because MongoDB is fast-booting and the Chatbot API establishes session history connections lazily, omitting this dependency speeds up synchronous API container boot times without compromising reliability.

1.5 Internal Docker DNS Service Discovery

To eliminate the fragility of hardcoded IP addresses, the architecture leverages Docker's internal DNS resolver. All containers reside within a default bridge network. Consequently, services map to each other using their logical container names:

- The Next.js API connects to the Redis session store natively via `redis://:$REDIS_PASSWORD`.
- The internal Elasticsearch sync scripts connect to the cluster via `http://elasticsearch`.
- The Chatbot API connects to MongoDB via `mongodb://$MONGO_USER:$MONGO_PASSWORD@mongodb`.

1.6 Environment Variable Strategy

The application employs a dual-file environment strategy to separate concerns:

- **`.env.local`:** Contains application-specific secrets (Supabase keys, API tokens) used by the Next.js application both during local development and within the Docker container.
- **`.env.docker`:** Contains Docker-specific infrastructure credentials (Redis password, MongoDB root credentials) referenced by `docker-compose.yml` for service configuration.

This separation ensures that infrastructure credentials are not leaked into the application runtime, and application secrets are not exposed to database containers.

1.7 Elasticsearch Configuration

The Elasticsearch container is explicitly configured for local development and integration within the microservices cluster. It is bootstrapped as a single node (`discovery.type=single-node`), removing the overhead of quorum elections. Furthermore, X-Pack security features are disabled (`xpack.security.enabled=false`) to streamline internal container-to-container communication without the friction of TLS and basic authentication management during the development phase. Finally, JVM memory limits are strictly enforced via environment variables (`ES_JAVA_OPTS=-Xms512m -Xmx512m`) to prevent the Java heap from starving other containers of host RAM, capping the Elasticsearch process at 1 GB total (512 MB initial + 512 MB maximum).

1.8 Technology Choice: Docker Compose vs Kubernetes

The platform employs Docker Compose for container orchestration rather than Kubernetes (K8s). While Kubernetes offers superior features for production-grade deployments including automatic horizontal pod autoscaling, self-healing via liveness/readiness probes, rolling updates with zero-downtime deployments, and declarative service mesh routing its operational complexity introduces significant overhead for a development-phase project of this scale.

Criteria	Docker Compose	Kubernetes
Configuration complexity	Single YAML file (145 lines)	Multiple resource manifests (Deployment, Service, ConfigMap, Ingress)
Learning curve	Minimal <code>docker compose up -d</code>	Requires <code>kubectl</code> , Helm charts, namespace management
Startup time	Seconds	Minutes (cluster bootstrapping, pod scheduling)
Health checks	Native <code>healthcheck</code> directive	Liveness/Readiness probes with finer granularity
Scaling	Manual (<code>docker compose scale</code>)	Automatic HPA based on CPU/memory metrics
Service discovery	Container name DNS (built-in)	CoreDNS with Service objects
Resource overhead	100 MB for Docker Engine	2 GB for control plane (etcd, API server, scheduler)

Docker Compose provides the ideal balance of simplicity and functionality for a seven-container development topology, while the architecture is designed to be Kubernetes-compatible for future production migration.

Chapter 2

Data Acquisition Pipeline (Web Scraping)

2.1 Overview

The Job Market Analytics Platform employs a robust, automated data acquisition pipeline designed to harvest, normalize, and persist job postings from major Vietnamese recruitment portals. The pipeline primarily targets TopCV and JobOKO, utilizing Playwright (headless Chromium) to handle JavaScript-heavy DOM rendering. This architecture is orchestrated via a GitHub Actions cron job running every three days (cron: `0 19 */3 * *`, corresponding to 02:00 AM Vietnam time), ensuring a continuous influx of fresh job data directly into the Supabase (PostgreSQL) data layer. The workflow also supports manual triggering via `workflow_dispatch` for on-demand data refresh.

2.2 Scraping Architecture and CI/CD Integration

The entry point for the data acquisition pipeline is `backend/jobs/github_action.ts`, which is triggered by the GitHub Actions cron schedule every three days. This automation ensures the platform's dataset remains current without requiring manual intervention.

2.2.1 CI/CD Dual-Runtime Pipeline

The GitHub Actions workflow orchestrates a multi-language pipeline, installing both **Node.js 22** (for the Playwright scrapers) and **Python 3.11** (for the offline normalization service) within a single CI run. The pipeline timeout is set to **150 minutes** to accommodate the sequential scraping of multiple sources with rate-limiting delays.

flowchart TD

```
CRON["GitHub Actions Cron<br/>(Every 3 Days, 02:00 VN)"] --> CHECKOUT["Step 1: Ch
CHECKOUT --> NODE["Step 2: Setup Node.js 22<br/>+ npm ci"]
NODE --> PYTHON["Step 3: Setup Python 3.11<br/>+ pip install"]
PYTHON --> PW["Step 4: Install Playwright<br/>Chromium + Deps"]
PW --> SCRAPE["Step 5: Run Scraper<br/>(npm run jobs:github)"]
SCRAPE --> NORM["Step 6: Normalize Data<br/>(python fix_khac_offline.py)"]
NORM --> DONE["Step 7: Pipeline Complete"]
```

```
subgraph "Step 5: Scraping Detail"
```

```
  S5A["JobOKO Scraper<br/>(2 pages)"]
```

```
  S5B["TopCV Playwright Scraper<br/>(1 page)"]
```

```
  S5C["Expired Jobs Cleanup"]
```

```
  S5D["N/A Link Verification<br/>(10 concurrent)"]
```

```
end
```

```
SCRAPE --> S5A
```

```
SCRAPE --> S5B
```

```
SCRAPE --> S5C
```

```
SCRAPE --> S5D
```

2.2.2 Technical Flow

1. **Initialization:** The process begins by validating Supabase Service Role Keys, establishing a secure connection that bypasses Row Level Security (RLS) policies for administrative ingestion.

2. **Scraping Sources:**

- **JobOKO** (scrapeJoboko): Configured dynamically via environment variables. The scraper delay is set to 3000ms in the CI environment (via `SCRAPER_DELAY_MS`),

though local development defaults to 7000ms for more conservative rate limiting. - **TopCV** (`scrapeTopCV`): Employs a more sophisticated, two-pass scraping pattern using Playwright to handle dynamic content (323 lines of scraper logic).

1. **Direct Upsert:** Data extracted by the scrapers is immediately persisted into the Supabase `jobs` table using an `upsert` operation, keyed on the unique job `url`. To optimize CI runtimes, data normalization is intentionally bypassed during the scraping phase and deferred to an offline Python script (`fix_khac_offline.py`). This script runs within the same CI job but as a separate step, because the FastAPI ML Service is not started in the CI environment it is intended for local development or future production deployment.

2.3 Playwright vs Traditional Scrapers

Modern recruitment portals like TopCV rely heavily on client-side JavaScript for rendering job listings dynamically. Traditional scraping tools (such as BeautifulSoup or Scrapy) operate on raw HTML and frequently fail to parse these dynamic Document Object Models (DOM). To overcome this, the platform utilizes Playwright's headless Chromium browser (`-no-sandbox`, `-disable-setuid-sandbox`), which executes the JavaScript and evaluates the DOM only after it has been fully rendered.

2.3.1 Technology Choice: Playwright vs Alternatives

Criteria	Playwright	Puppeteer	Selenium	Scrapy / BeautifulSoup
JS rendering	Full Chromium engine	Full Chromium engine	Full browser engines	None (static HTML only)
Language support	Node.js, Python, Java, C#	Node.js only	All major languages	Python only
Multi-browser	Chromium, Firefox, WebKit	Chromium only	Chrome, Firefox, Edge, Safari	N/A
CI/CD support	Native (<code>npx playwright install</code>)	Requires manual binary setup	Requires WebDriver binary management	Built-in (no browser needed)
Auto-wait	Built-in smart waits	Manual <code>waitForSelector</code>	Explicit <code>WebDriverWait</code>	N/A
Headless performance	200 MB per browser context	200 MB per browser context	300 MB per browser instance	20 MB (no browser)

Playwright was selected over Puppeteer for its superior multi-language support (enabling potential migration to Python scrapers in the future), native CI/CD integration, and built-in auto-wait mechanisms that simplify scraping JavaScript-rendered pages.

2.3.2 Anti-Scraping Countermeasures

To ensure reliable extraction and avoid IP bans, the `scrap_topcv.ts` implementation incorporates several evasion strategies:

- **User-Agent Spoofing:** The scraper simulates a legitimate user by mimicking a Chrome 120 browser operating on Windows 10 via the `User-Agent` header.
- **Accept-Language Header:** The scraper sets `Accept-Language: vi-VN,vi;q=0.9,en-US;q=0.5` signaling to the target server that the request originates from a Vietnamese-language browser. This reduces the likelihood of being flagged by geographic anomaly detection systems.
- **Delay Injection:** Aggressive rate limiting is applied. The system introduces an 8-second delay before querying listing pages to allow dynamic content to fully render. Between individual detail page crawls, a fixed 10-second delay is enforced to prevent triggering rate-limit thresholds on the target server.
- **Two-Pass Scraping:**

- *Pass 1 (Listing):* Extracts high-level metadata (title, URL, company name, salary, location, and logo) from search result pages, filtering out duplicate URLs in-memory using a JavaScript `Set`. - *Pass 2 (Detail):* Iterates over the deduplicated URLs, opens individual pages, waits for DOM readiness (2.5 seconds), and extracts deep content (full description, requirements, benefits, and expiration date) using heuristic DOM traversal techniques.

flowchart LR

```

subgraph "Pass 1: Listing Pages"
    LP["GET /viec-lam-tot-nhat?page=N"] --> WAIT1["delay(8000)<br/>Wait 8s for JS"]
    WAIT1 --> EVAL1["DOM evaluate<br/>Extract metadata"]
    EVAL1 --> DEDUP["In-memory Set<br/>URL deduplication"]
end

```

```

subgraph "Pass 2: Detail Pages"
  DEDUP --> OPEN["Open detail URL"]
  OPEN --> WAIT2["delay(2500)<br/>Wait 2.5s DOM ready"]
  WAIT2 --> EVAL2["DOM evaluate<br/>Extract description,<br/>requirements"]
  EVAL2 --> WAIT3["delay(10000)<br/>Wait 10s between pages"]
  WAIT3 --> OPEN
end

```

2.4 Concurrency Control and Data Freshness

Managing thousands of network requests requires strict concurrency limits to prevent connection timeouts and avoid overwhelming target servers.

- **Worker Pool Pattern (`runWithConcurrency`):** The pipeline utilizes a robust concurrency limit pattern, capping active workers at 10 concurrent requests for link checking. Workers independently request the next task from the pool upon completion, which is more efficient than batching with `Promise.all` and prevents isolated timeouts from stalling the entire queue. Each worker independently captures its own result (fulfilled or rejected), ensuring fault isolation.
- **Expired Jobs Cleanup:** To maintain data relevance, a maintenance job periodically fetches all records with explicit expiration dates (parsed from DD/MM/YYYY strings). These dates are compared against the current date, and expired records are purged in batches of 50 to minimize load on the Supabase Postgres instance. The batch size of 50 is chosen to comply with Supabase's `.in()` filter constraint, which limits the number of elements in a single filter predicate.
- **N/A Links Verification:** For job postings without an explicit expiration date (marked as 'N/A'), the system verifies the survival of the listing by dispatching HTTP requests to the URL via the `checkJobExists` function. Dead links are subsequently batch-deleted from the database in groups of 50.

2.5 Data Normalization Pipeline

While the primary scraping mechanism persists raw data directly to the database to optimize ingestion speed, the platform implements a robust, secondary offline processing pipeline to standardize and enrich this data. This normalization process is executed by a dedicated Python Machine Learning Service (334 lines), designed as a four-phase Extract, Transform, Load (ETL) pipeline.

2.5.1 Phase 1: Pattern-Based Cleansing

The initial phase addresses common structural anomalies inherent in scraped text. Utilizing targeted regular expressions, the system cleanses mandatory fields such as company names and geographical locations, stripping out irrelevant characters, errant whitespace, and inconsistent formatting to establish a baseline of clean string data.

2.5.2 Phase 2: Semantic Normalization and Feature Extraction

The core of the data enrichment process leverages rule-based heuristics.

- **Job Title Standardization:** Regular expressions are applied to isolate the core professional title by removing promotional prefixes (e.g., "Tuyn gp," "Cn tuyn") and descriptive suffixes detailing salary or urgency.
- **Categorization:** The pipeline maps diverse job listings into **66 pre-defined industry domains** (defined in the `CORE_DOMAINS` dictionary, spanning from "An toàn lao ng" to "Sinh viên / Thc tp sinh"). It utilizes a rule-based heuristic system which traverses the job title and description against a comprehensive keyword map of **99 keyword-to-domain mappings**.
- **Skill Extraction:** The system parses unstructured job descriptions to identify and extract up to ten specific professional skills. This is achieved using a rule-based dictionary of **64 professional skills** spanning office tools, programming languages, soft skills, and industry-specific competencies.

2.5.3 Phase 3: Deterministic Content Hashing

To facilitate absolute deduplication beyond simple URL collisions, the pipeline generates a deterministic hash identifier for each job. This hash is computed based on the normalized company name, the cleaned job title, and the standardized location, ensuring that identical postings from different source URLs are accurately identified and consolidated.

2.5.4 Phase 4: Persistence

In the final phase, the pipeline assembles the finalized payload, integrating the original fields with the newly computed normalized data and extracted skills. This enriched dataset is then persisted to the PostgreSQL data layer using an upsert operation, updating existing records and inserting new ones seamlessly.

2.5.5 Normalization Flow Diagram

The following diagram visualizes the flow of data through the four phases of the normalization pipeline:

```
flowchart LR
    R["Raw Job Record"] --> P1["Phase 1:<br/>Regex Cleansing"]
    P1 --> P2["Phase 2:<br/>Semantic Normalization"]

    subgraph "Phase 2 Detail"
        P2A["Title Regex Cleanup<br/>(Strip prefixes/suffixes)"]
        P2B["Rule-based Mapping<br/>(99 keywords 66 Domains)"]
        P2C["Rule-based Extraction<br/>(64 Skill Dictionary)"]
    end

    P2 --> P2A
    P2 --> P2B
    P2 --> P2C
    P2A --> P3["Phase 3:<br/>Content Hashing<br/>(SHA-256 Dedup)"]
    P2B --> P3
    P2C --> P3
    P3 --> P4["Phase 4:<br/>Supabase Upsert"]
```

2.6 Key Quantitative Metrics

| Metric | Value | |——|——-| | Scraper source files | 8 files, 77 KB total | | TopCV scraper complexity | 323 lines | | JobOKO scraper complexity | 516 lines | | Industry domains (Phase 2) | 66 categories | | Keyword-to-domain mappings | 99 entries | | Skill extraction dictionary | 64 professional skills | | CI/CD pipeline timeout | 150 minutes | | Cron schedule | Every 3 days at 02:00 VN | | Concurrency limit (link checking) | 10 workers | | Batch delete size | 50 records | | Listing page delay | 8,000 ms | | Detail page inter-crawl delay | 10,000 ms |

Chapter 3

Polyglot Persistence and Search Infrastructure

3.1 Overview

This chapter outlines the core data architecture of the Job Market Analytics Platform. Managing diverse data types ranging from highly structured user profiles to massive, unstructured job postings requires a specialized approach. The system adopts a polyglot persistence strategy, utilizing five distinct databases (Supabase, Redis, MongoDB, Elasticsearch, and Qdrant), each selected to optimally address specific data shapes and access patterns.

This chapter focuses on the three core platform components: **Supabase** (relational), **Redis** (caching and security), and **Elasticsearch** (full-text search). MongoDB and Qdrant are covered in the companion thesis (Thesis 1) as they primarily serve the AI Chatbot subsystem.

3.2 Polyglot Persistence Justification

The platform handles several distinct data workloads that impose conflicting requirements on a database system:

Workload	Access Pattern	Consistency Requirement	Optimal Database Type
User accounts & profiles	Relational joins, ACID transactions	Strong consistency	PostgreSQL (Supabase)
Job search queries	Full-text search, faceted aggregations	Eventual consistency	Elasticsearch
Session tokens & cache			Redis

Sub-millisecond reads, automatic expiry | Best-effort | Redis | | Chat conversation logs | Schema-less document appends | Eventual consistency | MongoDB | | Resume embeddings | Approximate nearest neighbor search | Eventual consistency | Qdrant |

3.2.1 Why Not a Monolithic Database?

A traditional, monolithic PostgreSQL approach was initially considered. However, the following trade-offs became evident:

Criteria	Monolithic PostgreSQL	Polyglot Architecture
Full-text search <code>ILIKE</code> / <code>tsvector</code> limited fuzzy search, no faceted aggregations	Elasticsearch BM25 with native facets	Session caching Disk-based I/O per session lookup Redis in-memory at sub-millisecond latency Vector similarity <code>pgvector</code> extension single-node, limited indexing algorithms Qdrant HNSW with named vector support
Schema flexibility Rigid schema migrations for chat logs MongoDB schemaless document appends	Operational complexity Single database, simple operations 5 databases, requires health checks and sync pipelines	

The polyglot approach trades operational complexity for optimized performance at each data access layer. The key architectural decision is to designate **Supabase as the single source of truth** and treat all other databases as **read-optimized replicas or caches**, following the Command Query Responsibility Segregation (CQRS) pattern.

3.3 Supabase (Core Relational Database)

Supabase, built on top of PostgreSQL, serves as the primary relational database for the application's core logic.

3.3.1 Structured Entities

Supabase is responsible for managing structured entities that require strong ACID (Atomicity, Consistency, Isolation, Durability) guarantees. The relational model ensures data integrity and enforces complex business rules through foreign key constraints.

erDiagram

```
auth_users ||--o| profiles : "has"
profiles ||--o{ experiences : "has"
profiles ||--o{ educations : "has"
profiles ||--o{ skills : "has"
profiles ||--o| user_cvs : "has"
profiles ||--o| user_resume_data : "has"
```

```
auth_users {
  uuid id PK
  string email
  jsonb user_metadata
}
```

```
profiles {
  uuid id PK
  string full_name
  string country
  string city
  timestamp updated_at
}
```

```
experiences {
  uuid id PK
  uuid user_id FK
  string company
  string role
  date start_date
  date end_date
}
```

```
educations {
  string id PK
  uuid user_id FK
  string school
  string degree
  string field_of_study
  string start_month
  string start_year
}
```



```

        string end_month
        string end_year
        string activities
        string description
        timestamp created_at
    }

    skills {
        uuid id PK
        uuid user_id FK
        string name
        string level
    }

    user_cvs {
        uuid id PK
        uuid user_id FK
        string file_name
        string file_path
        int file_size
        string file_type
    }

    jobs {
        bigint id PK
        string url UK
        string tieu_de
        string cong_ty
        string dia_diem
        string muc_luong
        string nganh_nghe_chuan_hoa
        jsonb ky_nang
        string job_hash_id
        jsonb thong_tin_tuyen_dung
        timestamp created_at
    }

```

3.3.2 Dual-Client Architecture

The application maintains two distinct Supabase client configurations to enforce access control separation:

- **Browser Client** (@supabase/supabase-js): Uses the public Anon Key, subject to Row Level Security (RLS) policies. Used for user-facing operations.
- **Server Client** (@supabase/ssr): Uses the Service Role Key, bypassing RLS. Used for administrative operations (scraper data ingestion, batch normalization).

3.4 Redis (Session, Cache, and Security Layer)

Redis is deployed as the high-speed, in-memory cache layer, serving three critical roles in the architecture: session management, query caching, and security enforcement.

3.4.1 Singleton Connection Pattern

Within the application, Redis connections are managed via a **global singleton pattern** (`globalForRedis`). This approach prevents connection multiplication during Next.js hot-module-reload (HMR) cycles in development, where module re-imports would otherwise create a new TCP connection per reload. In production (`NODE_ENV=production`), the singleton is not cached globally since HMR is not active.

3.4.2 Redis Key Space Architecture

The codebase employs structured key naming conventions with appropriate TTL strategies:

Key Pattern	Purpose	TTL	Module
<code>rate_limit:ip>window</code>	Fixed-window rate limiting	<code>windowSeconds + 10</code>	
<code>redisSecurity.ts</code>	JWT revocation on logout		
<code>Matches JWT exp claim</code>			
<code>redisSecurity.ts</code>			
<code>session:id</code>	Chatbot session state	24 hours	
<code>session_store.py</code>			
<code>session:id:history</code>	Chat history sync buffer	24 hours	
<code>session_store.py</code>			
<code>job:id</code>	Async CV processing job status	1 hour	
<code>job_tracker.py</code>			

3.4.3 Core Workloads

- **Authentication Security:** Redis handles both **JWT blacklisting** (SHA-256 hash with TTL matching token expiry) and **rate limiting** (Fixed Window Counter algorithm at 50 requests per 60 seconds per IP). These are detailed in Chapter 5.
- **Job Tracking:** Background processing tasks for CV extraction rely on Redis to track job state and progress with sub-millisecond updates.
- **Enum Caching:** Frequently accessed Elasticsearch aggregation results (cities, experience buckets, work types, categories) are cached in the server's RAM via the **EnumCache** class, with periodic async refresh from Elasticsearch every 3,600 seconds.

3.5 Elasticsearch (Full-Text Search Engine)

To power the core "job search" functionality, the system relies on Elasticsearch 8.13.0. Standard relational databases are not optimized for unstructured text search at scale.

3.5.1 CQRS Data Flow

flowchart LR

```
SCRAPER["Playwright Scrapers"] -->|"upsert (raw data)"| SUPA["Supabase<br/>(Source of Truth)"]
NORMALIZER["Python ML Service"] -->|"upsert (normalized)"| SUPA
SUPA -->|"npm run es:sync<br/>(batch 500 records)"| SYNC["sync.ts<br/>Transform + Sync"]
SYNC -->|"Bulk API<br/>(refresh: true)"| ES["Elasticsearch<br/>(Read Index)"]

FRONTEND["Next.js Frontend"] -->|"GET /api/v1/jobs/search"| API["API Route"]
API -->|"bool query"| ES
CHATBOT["AI Chatbot"] -->|"search_jobs tool"| ES
```

3.5.2 Data Synchronization

Job posting data is synchronized from Supabase into Elasticsearch via a batch pipeline (`npm run es:sync`). The sync script employs a **full-rebuild**

strategy: it deletes and recreates the entire index on each run, then re-indexes all records. While this approach is less efficient than incremental sync, it guarantees index consistency and eliminates stale-document accumulation. The pipeline processes records in paginated batches of 500, ordered by `created_at` descending (newest first), and applies in-memory URL-based deduplication via a JavaScript `Set`.

3.5.3 Advanced Mapping and Indexing

Elasticsearch indices are carefully mapped to support diverse search requirements:

Field	ES Type	Analyzer	Purpose
<code>url</code>	<code>keyword</code>		Unique document ID
<code>tieu_de</code>	<code>text</code>	<code>standard</code>	Job title full-text search (boosted E3)
<code>cong_ty</code>	<code>text</code>	<code>standard</code>	Company name search (boosted E2)
<code>cities</code>	<code>keyword</code>		Location faceted filter (65 geographic entities)
<code>categories</code>	<code>keyword</code>		Industry domain faceted filter (66 domains)
<code>workTypes</code>	<code>keyword</code>		Work type faceted filter
<code>levels</code>	<code>keyword</code>		Seniority level faceted filter
<code>expBuckets</code>	<code>keyword</code>		Experience tier faceted filter (4 ranges)
<code>salaryBuckets</code>	<code>keyword</code>		Salary tier faceted filter (6 ranges)
<code>created_at</code>	<code>date</code>		Temporal ordering
<code>raw_data</code>	<code>object</code>		(enabled: <code>false</code>) Full job document stored but NOT indexed

The `raw_data` field uses `enabled: false` to store the original job JSON without indexing its internal contents. This significantly saves system resources (RAM, CPU, disk) during indexing while retaining all data for retrieval the frontend reads job details directly from `raw_data` without requiring a secondary query to Supabase.

3.6 Technology Choice: Elasticsearch vs Alternatives

Criteria	Elasticsearch 8.13	Typesense	Meilisearch	PostgreSQL Full-Text
Relevance algorithm	BM25 (default since v5)	TF-IDF variant	Proximity-based	<code>ts_rank</code> with <code>tsvector</code>
Faceted aggregations	Native <code>terms</code> aggregations	Native facets	Native facets	<code>GROUP BY</code> (expensive at scale)
Fuzzy search	Levenshtein edit distance (<code>fuzziness: AUTO</code>)	Typo		

tolerance (1-2 chars) | Typo tolerance (built-in) | **ILIKE** / trigram similarity
 | | **Vietnamese NLP** | Standard analyzer (tokenization, lowercasing) | No
 built-in CJK support | No built-in CJK support | **simple/pg_trgm** dictio-
 naries | | **Deployment** | Docker official image | Single binary | Single binary
 | Built into Supabase | | **Scalability** | Horizontal sharding | Vertical scaling
 | Vertical scaling | Connection-limited | | **Ecosystem** | Mature (REST API,
 Kibana) | Lightweight | Lightweight | Native PostgreSQL | | **Memory usage**
 | 512 MB (JVM heap) | 50 MB | 50 MB | Shared with main DB |

Elasticsearch was selected for its mature ecosystem, native Docker sup-
 port, powerful **bool** query DSL, and production-proven aggregation capabil-
 ities. Its BM25 relevance scoring provides superior search quality compared
 to PostgreSQL's basic full-text capabilities, which is critical for a job search
 platform processing thousands of Vietnamese-language postings.

3.7 Key Quantitative Metrics

| Metric | Value | |——|——| | Total databases in polyglot architecture
 | 5 (Supabase, Redis, Elasticsearch, MongoDB, Qdrant) | | Core databases
 (Thesis 2 scope) | 3 (Supabase, Redis, Elasticsearch) | | Supabase tables | 7
 (**profiles, experiences, educations, skills, user_cvs, jobs, user_resume_data**)
 | | Redis key patterns | 5 distinct naming conventions | | Redis rate limit
 threshold | 50 requests / 60 seconds / IP | | EnumCache TTL | 3,600 seconds
 (1 hour) | | ES index fields | 10 mapped fields + 1 stored-only (**raw_data**) |
 | ES sync batch size | 500 records per Supabase fetch | | ES JVM heap allo-
 cation | 512 MB initial / 512 MB maximum | | Geographic entities (cities)
 | 65 entries (63 provinces/regions + "Toàn quc" + "Nc ngoài") | | Industry
 domain categories | 66 standardized categories | | Salary buckets | 6 ranges
 (03, 35, 510, 1020, 2050, 50+ triu VND) | | Experience buckets | 4 ranges (<
 1, 12, 25, 5+ years) |

Chapter 4

Full-Text Search Engine (Elasticsearch)

4.1 Search Architecture & Rationale

Building a job search system requires the ability to handle complex text queries, faceted filtering, and fast response times. In the initial phase, a relational database system (PostgreSQL via Supabase) was used as the primary data source. However, as the data volume grew and search requirements became more complex, this architecture revealed several limitations:

- **Limitations of SQL:** Full-text search queries or `ILIKE` operators in PostgreSQL are not optimized for natural language search, especially when handling spelling errors and relevance scoring. PostgreSQL's `tsvector` / `tsquery` system lacks native Vietnamese language support and provides limited relevance tuning compared to specialized search engines.
- **Faceted Aggregations:** Dynamically calculating filter categories (e.g., counting the number of jobs per city or salary range) consumes a lot of resources if executed directly on SQL using complex `GROUP BY` clauses. Each unique filter combination requires a separate query, leading to N+1 aggregation overhead.

To resolve these issues, the system architecture adopts the Command Query Responsibility Segregation (CQRS) pattern:

- **Supabase (PostgreSQL)** acts as the "Source of Truth" (Transactional Database), storing the original data after scraping and normalization.
- **Elasticsearch 8.13** is used as a "Read-only Index" (Analytical Database), dedicated to processing search queries and filtering data for the frontend and AI Chatbot. The Elasticsearch system is deployed as a single-node cluster via Docker (`elasticsearch:8.13.0` on port 9200), ensuring lightning-fast search capabilities and multi-language support.

4.2 Index Schema & Mapping Strategies

To optimize Elasticsearch for both relevance scoring and exact filtering, the index structure (`jobs`) is meticulously designed within the data synchronization configuration file.

Two main data type strategies are used:

1. **Keyword Mapping:** Applied to fields that require exact match filtering and aggregations, such as `url`, `cities` (65 geographic entities), `categories` (66 industry domains), `workTypes`, `levels`, `expBuckets` (4 tiers), and `salaryBuckets` (6 ranges). The `keyword` type allows Elasticsearch to build optimal data structures (inverted index on exact terms) for directly searching original text structures without tokenization.
2. **Text Mapping:** Applied to fields that require relevance scoring, such as `tieu_de` (job title) and `cong_ty` (company name), using the default `standard` analyzer. The standard analyzer performs Unicode-aware tokenization, lowercasing, and stop word removal, enabling BM25-based relevance ranking.

Optimizing the "Document Store" with `enabled: false`: Instead of extracting every detail of the job for indexing, the `raw_data` field is configured as an `object` type with the `enabled: false` option. This option instructs Elasticsearch to store the original JSON string of the job without indexing its internal contents. This approach significantly saves system resources (RAM, CPU, disk space) during the indexing process while retaining all the data. When a search query returns results, the frontend can immediately retrieve detailed job data from `raw_data` without needing an additional query to Supabase for the information.

4.3 Data Ingestion & Deduplication Pipeline (Sync Flow)

Data is updated from Supabase to Elasticsearch via a batch synchronization pipeline. This process is triggered by a Node.js script (`sync.ts`).

The sync pipeline employs a **full-rebuild strategy**: the existing index is deleted and recreated on each run, ensuring zero stale-document accumulation. While this is less efficient than incremental sync for very large datasets, it guarantees index consistency and eliminates the need for complex change-detection logic.

flowchart TD

```
START["npm run es:sync"] --> DELETE["Delete existing<br/>'jobs' index"]
DELETE --> CREATE["Create new index<br/>with mappings"]
CREATE --> FETCH["Fetch from Supabase<br/>(batch of 500,<br/>newest first)"]
FETCH --> DEDUP["In-memory Set dedup<br/>(URL or title-company<br/>composi"]
DEDUP --> TRANSFORM["toEsDoc() transform<br/>(splitLocations, getSalaryBuc"]
TRANSFORM --> BULK["Elasticsearch Bulk API<br/>(refresh: true)"]
BULK --> CHECK{"More batches?"}
CHECK -->|"data.length == 500"| FETCH
CHECK -->|"data.length < 500"| DONE["Sync Complete"]
```

1. **Batch Fetching:** The system fetches data from Supabase in paginated batches (500 records per batch) ordered by `created_at` descending (newest first). Breaking the data down into batches helps prevent memory overflow.
2. **Memory-Resident Deduplication:** To avoid data duplication before pushing to Elasticsearch, an in-memory **Set** data structure (RAM) is used to track records that have already appeared. The primary deduplication key is the job URL. If the URL is missing, a composite key in the format `$tieu_de-$cong_ty` (title-company) is used as the fallback identifier.
3. **Normalization Transform:** Each job record passes through five normalization functions (`splitLocations`, `getSalaryBuckets`, `getExpBuckets`, `getWorkTypeTags`, `getLevels`) that convert unstructured Vietnamese text into standardized keyword facets.

4. **Bulk API:** The deduplicated and normalized data is pushed using the `_bulk` API, which exponentially improves write speeds compared to sequentially writing individual documents. These operations are executed with the `refresh: true` option to ensure that new data is searchable immediately after the sync completes.

4.4 Vietnamese NLP & Heuristic Normalization Logic

Due to the unstructured nature of Vietnamese job postings, an internal library (`helpers.ts`, 152 lines) was built to extract and normalize data through Heuristic and Regex techniques prior to indexing. These exact same parsing heuristics are shared with the client-side frontend routing layer (as detailed in Chapter 6, Section 6.4) to ensure that browser-evaluated filter state matches the backend index representations precisely:

- **Location Parsing (`splitLocations`):** Raw location strings (e.g., "Ni làm vic: Hà Ni, Tp. HCM") are stripped of unnecessary prefixes, split by commas, and matched against a static array of **65 geographic entities** (`CITY_PATTERNS`). The system applies Regex for noise filtering when job title keywords (such as "chuyên viên", "trng phòng") are mistakenly identified as locations. Notably, the parser implements a **negative lookbehind regex** (`(?!Bà Ra) - (?!Vng Tàu)`) to prevent incorrectly splitting the compound province name "Bà Ra - Vng Tàu" on the hyphen delimiter – a critical edge case in Vietnamese geographic parsing. Additionally, the `CITY_PATTERNS` array explicitly includes both "Vng Tàu" and "Bà Ra - Vng Tàu" to ensure maximum flexibility: search queries containing only "Vng Tàu" resolve correctly to the city's listings, while provincial queries map to "Bà Ra - Vng Tàu".
- **Salary Bucketing (`getSalaryBuckets`):** Foreign currency salaries are filtered out using a comprehensive regex matching **17 international currency codes** (USD, EUR, GBP, JPY, SGD, AUD, CAD, HKD, KRW, THB, MYR, INR, CNY, RMB, TWD, CZK, CHF). For VND salaries, the system uses Regex to extract number ranges (e.g., "10 - 15 triu"), removes commas, converts them to floating-point decimals representing millions of VND, and classifies them into 6 fixed income ranges: 03, 35, 510, 1020, 2050, and over 50 million VND.

- **Experience Parsing (getExpBuckets):** Text regarding experience (e.g., "di 1 nm", "trên 5 nm", or ranges like "1 - 2 nm") is scanned and normalized into a time range value in years. Month-based values (e.g., "2-5 tháng") are automatically converted to years by dividing by 12. It is then assigned to 4 static experience buckets: Under 1 year, 12 years, 25 years, Over 5 years.
- **Work Types (getWorkTypeTags):** English slang or mixed Vietnamese keywords (e.g., "full-time", "part-time", "thi v", "wfh", "hybrid") are mapped to 5 standard tags: "Toàn thi gian" (Full-time), "Bán thi gian" (Part-time), "Thc tp" (Internship), "Thi v" (Contract), "Làm ti nhà" (Remote/WFH).

4.5 Multi-Field Search & Relevance Tuning

When a search command is received from the AI Chatbot or the frontend API, the query is constructed via the Python async Elasticsearch client (`data_clients.py`) using the bool Query DSL.

flowchart TD

```
QUERY["User Search Query"] --> BOOL["bool Query"]
```

```
subgraph "must clause (Relevance Scoring)"
```

```
MM["multi_match<br/>type: best_fields<br/>fuzziness: AUTO"]
```

```
MM --> F1["tieu_de ^3<br/>(title, weight E3)"]
```

```
MM --> F2["cong_ty ^2<br/>(company, weight E2)"]
```

```
end
```

```
subgraph "filter clause (Exact Match)"
```

```
FLT1["terms: cities"]
```

```
FLT2["terms: categories"]
```

```
FLT3["terms: levels"]
```

```
FLT4["terms: expBuckets"]
```

```
FLT5["terms: workTypes"]
```

```
FLT6["terms: salaryBuckets<br/>(auto-resolved from range)"]
```

```
end
```

```
BOOL --> MM
```

```
BOOL --> FLT1
BOOL --> FLT2
BOOL --> FLT3
BOOL --> FLT4
BOOL --> FLT5
BOOL --> FLT6
```

```
BOOL --> SORT["Sort: _score, created_at desc"]
SORT --> SOURCE["_source: raw_data only"]
```

1. **Relevance Scoring (Must Clause):** If the user provides a **keyword**, the query uses the **multi_match** command with the **best_fields** option. The job title (**tieu_de**) is weighted with a multiplier of 3 (~ 3), while the company name (**cong_ty**) receives a weight multiplier of 2 (~ 2). The **fuzziness: AUTO** feature enables Levenshtein edit-distance tolerance, allowing 0 edits for terms 12 characters, 1 edit for 35 characters, and 2 edits for 6+ characters. This handles common typos but is not Vietnamese-specific NLP it operates at the character level.
2. **Strict Filtering (Filter Clauses):** Optional parameters such as location, salary, and experience are passed in as **terms** filters, instructing the engine to strictly match the content against the defined **keyword** types. Filter clauses do not affect relevance scoring and are cached by Elasticsearch for performance.
3. **Salary Resolution:** When given an input range for desired salary (**min_salary** to **max_salary**), the algorithm automatically identifies all salary buckets that overlap with the request and injects them into the query as a list of valid keywords. For example, a request for 815 million VND would resolve to buckets "5 10 triu" and "10 20 triu".
4. **City Prioritization:** If a user searches by location, the returned payload within **raw_data** is automatically adjusted by moving the queried city to the top of the **cities** list. This reordering helps the frontend interface and AI better assess visual priority when displaying multiple locations.
5. **Source Filtering:** The **_source** field is restricted to ["**raw_data**"] only, avoiding the transfer of redundant indexed fields in the response payload.

4.6 Aggregation & Memory-Resident Enum Caching

To build a UI filtering experience or to provide a valid set of parameters for the AI Chatbot's Tool Calling process, the system needs to know the available filter values (e.g., which cities currently have job postings).

- **Elasticsearch Aggregations:** The client sends queries with a size of 0 (`size: 0`) requesting `terms aggregations` to group distinct attributes (`distinct_cities` up to 1,000 buckets, `distinct_categories` up to 500 buckets).
- **EnumCache Pattern:** To avoid constantly sending Aggregation queries that consume Elasticsearch resources, the `EnumCache` class (`enum_cache.py`, 115 lines) maintains these lists directly in the server's memory (RAM) with a Time-To-Live (TTL) of 3,600 seconds (1 hour).
- **Async Background Refresh:** The cache refresh process runs as a background task via `asyncio.Task(_refresh_loop)`. Therefore, any GET request from the frontend or Pydantic Validator from the Chatbot can retrieve these data tags in sub-millisecond times thanks to synchronous properties (`@property`), significantly improving the application's overall response speed.

4.7 Key Quantitative Metrics

Metric Value	ES Docker image version <code>elasticsearch:8.13.0</code>
JVM heap allocation 512 MB initial / 512 MB maximum	Sync batch size 500 records per Supabase fetch
City patterns dictionary 65 geographic entities	Industry domain categories 66 standardized tags
Salary buckets 6 ranges (VND millions)	Experience buckets 4 ranges
Work type tags 5 standard categories	Foreign currency filter 17 international currency codes
Title boost weight $\text{E}3 (\text{tieu_de}^3)$	Company boost weight $\text{E}2 (\text{cong_ty}^2)$
EnumCache TTL 3,600 seconds (1 hour)	Max aggregation buckets (cities) 1,000
Max aggregation buckets (categories) 500	helpers.ts complexity 152 lines
enum_cache.py complexity 115 lines	

Chapter 5

Authentication & Authorization

5.1 Architecture Overview

In the CareerIntel platform, securing user data and managing access control are pivotal. Instead of building and maintaining a custom JSON Web Token (JWT) issuance system from scratch, the project leverages a managed authentication model via **Supabase Auth**. Supabase provides a robust identity solution, backed by PostgreSQL, which integrates tightly with the Next.js 16 Middleware architecture.

To add an additional security layer, the platform integrates **Redis** to handle Token Revocation (Blacklisting) immediately when a user logs out, and **Rate Limiting** to protect API endpoints from abuse. The combination of Supabase (for identity verification) and Redis (for session state control and security enforcement) creates a system that is both secure and highly flexible.

5.1.1 Technology Choice: Supabase Auth vs Alternatives

Criteria	Supabase Auth	Auth0	Firebase Auth	Custom JWT	
	— ————— — — ————— —————	— ————— — — ————— —————	— ————— — — ————— —————	— ————— — — ————— —————	
					Hosting Managed (PostgreSQL-backed) Managed SaaS Managed (Google Cloud) Self-hosted Pricing Free tier (50K MAU) Free tier (7.5K MAU) Free tier (10K verifica-

tions/month) | Infrastructure cost only | | **Next.js integration** | @supabase/ssr with cookie management | @auth0/nextjs-auth0 | firebase-admin SDK | Manual cookie/header management | | **Database integration** | Native PostgreSQL RLS policies | External user store | Firestore integration | Custom user tables | | **Social login** | Google, GitHub, OAuth | 30+ social providers | Google, Facebook, Twitter | Manual OAuth implementation | | **JWT customization** | Limited (via PostgreSQL hooks) | Extensive (Actions/Rules) | Limited | Full control | | **RLS (Row Level Security)** | Native support | Not applicable | Firestore Security Rules | Manual implementation | | **Operational overhead** | Minimal | Minimal | Minimal | High (key rotation, revocation, migration) |

Supabase Auth was selected for its **native PostgreSQL Row Level Security (RLS) integration**, which allows database-level access control policies without custom middleware. Since the application already uses Supabase for its relational data layer, using its built-in auth system eliminates the need for a separate identity provider and reduces integration complexity.

5.2 Authentication Flow

The authentication process is designed around Server-side Cookies rather than storing tokens in the browser's Local Storage. This approach helps mitigate Cross-Site Scripting (XSS) attacks aimed at token theft.

1. **Login:** The user provides an email and password. The frontend calls a Server Action (utilizing Supabase's `signInWithPassword`).
2. **Token Issuance:** Supabase verifies the credentials and returns a JWT. This JWT is set into an HTTP-only Cookie by the Server Client.
3. **Request Validation:** Whenever the browser makes a request to protected pages, the Next.js Middleware reads the Cookie and validates the JWT using `supabase.auth.getUser()`.
4. **Logout:** When the user logs out, the current JWT is extracted and hashed, then stored in the Redis Blacklist with a Time-To-Live (TTL) corresponding to the token's remaining expiration time. Finally, the Supabase `signOut` function is called to clear the Cookie.

```
sequenceDiagram
    participant User
    participant Browser
    participant ServerAction as Next.js Server Action
    participant Middleware as Next.js Middleware
    participant Supabase as Supabase Auth
    participant Redis as Redis (Blacklist)

    User->>Browser: Login (email + password)
    Browser->>ServerAction: Call Server Action (login)
    ServerAction->>Supabase: signInWithPassword()
    Supabase-->>ServerAction: JWT Token
    ServerAction-->>Browser: Set JWT in HTTP-only Cookie
    Browser->>Middleware: GET /profile (With Cookie)
    Middleware->>Supabase: getUser() (Validate JWT)
    Supabase-->>Middleware: User Object
    Middleware-->>Browser: Allow Access

    User->>Browser: Logout
    Browser->>ServerAction: Call Server Action (logout)
    ServerAction->>Redis: blacklistToken(JWT)
    ServerAction->>Supabase: signOut() (Clear Cookie)
    ServerAction-->>Browser: Redirect to /login
```

5.3 Route Protection Strategy

The platform's routing system is divided into two primary groups: Public Routes and Protected Routes. Access control is enforced in the `middleware.ts` file using the `createServerClient` from the `@supabase/ssr` library. The middleware intercepts every incoming request and validates the JWT before allowing access.

- **Protected Pages:** Pages such as `/profile`, `/ai`, `/search`, `/insights`, and `/job` (including all nested routes such as `/job/[id]`) strictly require a valid session. The `startsWith()` matching ensures all child routes inherit the protection policy.
- **Public vs Protected APIs:** Certain APIs like `/api/chatbot` and

`/api/kie` are publicly accessible, whereas all other `/api/*` routes require authentication.

Handling Unauthenticated Requests: The system handles requests intelligently based on their context.

- If a user attempts to access a protected page (UI) without a token, the Middleware responds with a **307 Redirect** to the `/login` page.
- However, if a request is made to a Protected API (for example, fetching personal data via a `fetch` call), a redirect is inappropriate and would cause client-side errors. In this scenario, the Middleware returns a **401 Unauthorized** status code in JSON format, allowing the frontend to catch and handle the error gracefully.

5.4 Advanced Security: JWT Blacklisting via Redis

An inherent drawback of Stateless JWT architecture is that a token cannot be revoked before its expiration time. To address this, the project implements a **JWT Blacklisting** mechanism using Redis within the `redisSecurity.ts` module.

5.4.1 Optimized Storage Process:

Instead of storing the raw JWT string (which may contain sensitive payloads and consume more storage), the system employs the **SHA-256** algorithm to generate a Hash from the JWT via the `getTokenHash()` function. This hash serves as the key stored in Redis. Furthermore, the key's Time-To-Live (TTL) in Redis is calculated based on the JWT's `exp` claim, by subtracting the current time from the expiration time (`getJwtExpiry()`). Consequently, Redis automatically frees up memory once the token expires, eliminating the need for manual garbage collection.

flowchart LR

```
JWT["JWT Token"] --> DECODE["Decode payload<br/>(Base64)"]
DECODE --> EXP["Extract 'exp' claim"]
JWT --> HASH["SHA-256 Hash"]
```



```

HASH --> KEY["Redis Key:<br/>blacklist:{hash}"]
EXP --> TTL["TTL = exp - now()"]
KEY --> SET["redis.set(key, 'revoked',<br/>'EX', TTL)"]
TTL --> SET
SET --> AUTO["Auto-expiry when<br/>JWT naturally expires"]

```

5.4.2 Fail-Closed vs Fail-Open: Deliberate Security Trade-offs

The `redisSecurity.ts` module implements two distinct failure strategies depending on the security criticality of the operation:

Function	Failure Strategy	Rationale
<code>isTokenBlacklisted()</code>	Fail-Closed	(treat as blacklisted) Security > Availability prevents revoked tokens from slipping through if Redis is unreachable
<code>checkRateLimit()</code>	Fail-Open	(allow request through) Availability > Security rate limiting failure should not block legitimate search requests

This deliberate contrast ensures that authentication-critical operations prioritize security (forcing affected users to re-login temporarily), while non-critical operations like rate limiting prioritize availability (allowing requests through with degraded protection).

5.5 Rate Limiting

In addition to JWT blacklisting, the `redisSecurity.ts` module implements **API Rate Limiting** using the **Fixed Window Counter** algorithm.

5.5.1 Mechanism:

1. A Redis key is constructed from the client's IP address and the current time window: `rate_limit:ip:floor(now / windowSeconds)`.
2. Each incoming request atomically increments the key using `redis.incr(key)`.
3. If the key is newly created (count = 1), a TTL of `windowSeconds + 10` seconds is set to ensure automatic cleanup.

4. If the count exceeds the configured limit (default: **50 requests per 60 seconds**), the request is rejected.

5.5.2 Design Decisions:

- **Fixed Window vs Sliding Window:** The Fixed Window algorithm was chosen for its simplicity and minimal Redis operations (single INCR per request). While it can allow up to 2x burst at window boundaries (a known limitation), this trade-off is acceptable for the platform's traffic patterns.
- **TTL Buffer (+10s):** The extra 10 seconds on the TTL ensures that the Redis key outlives the window, preventing a race condition where the key expires mid-window and resets the counter.

5.6 Form Management and Input Validation

The processing of authentication forms (Login, Signup, Profile Update) is handled through the **Next.js Server Actions** ('use server') paradigm. This approach completely eliminates the need to create intermediary API routes solely for receiving form data, thereby reducing network complexity and improving processing speed.

All input data is strictly validated using the **Zod** library (**LoginSchema**, **SignupSchema**). Actions that record profile data (Experience, Education, Skill, CV) into public PostgreSQL tables (via the Supabase client) are tightly bound to the current **user_id**. This ensures data integrity and prevents Insecure Direct Object Reference (IDOR) vulnerabilities – a user can only create, update, or delete records where **user_id** matches their authenticated session.

5.7 Key Quantitative Metrics

Metric	Value
Protected page routes	5 (/profile, /ai, /search, /insights, /job)
Public API routes	2 (/api/chatbot, /api/kie)
Rate limit threshold	50 requests / 60 seconds / IP
Rate limit algorithm	Fixed Window Counter
JWT hash algorithm	SHA-256
Blacklist TTL	Dynamic (matches JWT exp claim)
Default fallback TTL	

| 3,600 seconds (1 hour) | | Rate limit TTL buffer | +10 seconds beyond window | | Auth validation schemas | 2 (LoginSchema, SignupSchema) | | Profile CRUD tables | 5 (**profiles**, **experiences**, **educations**, **skills**, **user_cvs**) | | redisSecurity.ts complexity | 126 lines | | middleware.ts complexity | 59 lines | | actions.ts complexity | 297 lines (12 Server Actions) |

Chapter 6

Core Frontend Layer (Next.js)

6.1 Overview

The core frontend of the CareerIntel platform is constructed using the Next.js 16 App Router architecture. This layer serves as the primary gateway for users, offering a cohesive application shell, robust job search interfaces, and comprehensive profile management tools. This chapter details the foundational frontend architecture, focusing on server-side rendering strategies for authentication, the URL-driven state of faceted search interfaces, and the offloading of Vietnamese Natural Language Processing (NLP) to the client.

6.1.1 Technology Choice: Next.js App Router vs Alternatives

Criteria	Next.js App Router	Next.js Pages Router	React + Vite (SPA)
Nuxt.js (Vue)			
Rendering model	Hybrid SSR + Client	SSR via <code>getServerSideProps</code>	Client-only (CSR)
Server Components	Native (React Server Components)	Not supported	Not applicable
Auth data fetching	Server-side cookie access in RSC	<code>getServerSideProps</code> only	Client-side only (flash of unauth content)
Route protection	<code>middleware.ts</code> edge function	<code>_middleware.ts</code> (experimental)	Manual route guards
API layer	File-based API routes in <code>app/api/</code>	File-based in <code>pages/api/</code>	Separate backend required

tory | | **Server Actions** | Native 'use server' for forms | Not supported | Not applicable | `useAsyncData` with API calls | | **Streaming/Suspense** | Native `loading.tsx` + `Suspense` | Manual implementation | Not applicable | `<Suspense>` support | | **Bundle splitting** | Automatic per-route | Manual dynamic imports | Manual lazy imports | Automatic per-route |

Next.js App Router was selected for its native Server Components support, which enables secure server-side authentication cookie handling, eliminates the "flash of unauthenticated content" common in SPAs, and provides built-in Server Actions that remove the need for intermediary API routes for form submissions.

6.2 SSR Data Fetching and User-Aware Rendering

Next.js 16 introduces a clear delineation between Server Components and Client Components. The application leverages Server Components as the default execution environment, heavily optimizing Time to Interactive (TTI) and securing the authentication flow.

To render personalized content based on the user's authentication state, Server-Side Rendering (SSR) data fetching is deeply integrated with Supabase. The application utilizes a server-side Supabase client instantiated via `createClient()`. This client intercepts the incoming HTTP request in the Node.js runtime, reads the secure, HttpOnly JWT cookies, and validates the user session before emitting any HTML to the browser. This architectural choice allows protected routes, such as the `/profile` page, to be fully hydrated with the user's specific data (experiences, education, skills) on the server. This eliminates the "flash of unauthenticated content" typical of Single Page Applications (SPAs) and enhances security by preventing client-side interception of raw tokens.

6.3 Faceted Search Architecture

The job search interface, located at `/search`, provides users with an Elasticsearch-powered faceted search experience. Managing the complex state of numerous interdependent filters requires a robust state management strategy.

6.3.1 Hybrid State Management Strategy

The search interface implements a hybrid state management model to balance initial deep-linking capability with high-speed, client-side responsiveness.

1. **Initial Parameter Hydration:** On component mount, the search page utilizes Next.js's `useSearchParams()` hook to read initial search criteria (e.g. `keyword`, `location`, or `category`) directly from the browser's URL. This allows deep-linking to search pages (such as user redirection from landing page query boxes).
2. **Local React State Control:** For subsequent filtering interactions, the state is managed locally via React's `useState` hook (`filters` state). When a user interacts with the custom `DropdownFilter` components to select locations, levels, or salary ranges, the component updates this React state rather than immediately pushing mutations to the browser's address bar history, avoiding history-stack pollution and route re-evaluations.
3. **Reactive Fetching Loop:** A `useEffect` hook monitors the `filters` state. When updated, it compiles the selected facets into a query parameter string and dispatches an asynchronous `fetch` request to the backend search handler `/api/v1/jobs/search?locations=....`

sequenceDiagram

```
participant User
participant FilterUI as DropdownFilter
participant PageState as React State (useState)
participant ClientHook as useEffect
participant ES_API as /api/v1/jobs/search
participant ES as Elasticsearch
```

```
User->>FilterUI: Select "Hà Ni"
```

```
FilterUI->>PageState: Update filters state (filters.locations = ["Hà Ni"])
```

```
PageState->>ClientHook: Trigger fetchJobs callback
```

```
ClientHook->>ES_API: GET /api/v1/jobs/search?locations=Hà Ni
```

```
ES_API->>ES: bool query (filter: cities=["Hà Ni"])
```

```
ES-->>ES_API: Matching job documents
```

```
ES_API-->>PageState: Set job list state (setJobs)
```

```
PageState-->>FilterUI: Re-render Search Results (React Virtual DOM)
```

This hybrid approach allows the interface to maintain linkability upon entry while ensuring subsequent multi-criteria queries are completed instantly within the page scope without browser URL changes.

6.3.2 Supported Filter Parameters

URL Parameter	ES Field	Type	Example
<code>q</code>	<code>tieu_de, cong_ty</code>	<code>multi_match</code> (keyword)	<code>?q=developer</code>
<code>locations</code>	<code>cities</code>	terms filter	<code>?locations=Hà Ni</code>
<code>categories</code>	<code>categories</code>	terms filter	<code>?categories=IT Phn mm</code>
<code>levels</code>	<code>levels</code>	terms filter	<code>?levels=Qun lý</code>
<code>expBuckets</code>	<code>expBuckets</code>	terms filter	<code>?expBuckets=2 5 nm</code>
<code>salaryBuckets</code>	<code>salaryBuckets</code>	terms filter	<code>?salaryBuckets=10 20 triu</code>
<code>workTypes</code>	<code>workTypes</code>	terms filter	<code>?workTypes=Toàn thi gian</code>
<code>page</code>	ES from	Pagination offset	<code>?page=2</code>

6.4 Client-Side Vietnamese NLP Processing

While the core Machine Learning normalization pipeline processes job data asynchronously on the backend, the frontend implements lightweight, deterministic Natural Language Processing (NLP) tailored for the Vietnamese language. As detailed in Chapter 4, Section 4.4, these client-side parsers share the exact same heuristic and regex structures as the backend synchronization library helpers.ts to ensure consistency between the indexed facets and browser-evaluated filter state. This offloading strategy reduces API overhead and allows for instantaneous client-side formatting and data binning.

6.4.1 Location and Geographic Parsing (`splitLocations`)

Job postings on Vietnamese portals frequently concatenate multiple geographic regions into unstructured strings (e.g., "Khu vc: H Chí Minh, Hà Ni - à Nng"). The frontend implements a `splitLocations` parser utilizing a predefined dictionary of **65 valid geographic entities** (63 Vietnamese provinces/regions plus "Toàn quc" and "Nc ngoài") and localized regex patterns.

The parser performs the following operations:

1. **Prefix stripping:** Removes common Vietnamese location prefixes ("Ni làm vic:", "Khu vc:", "Ti:").
2. **Intelligent delimiter splitting:** Splits on commas and semicolons. For hyphens, a **negative lookbehind regex** (`(?<!Bà Ra) - (?!Vng Tàu)`) is employed to prevent incorrectly splitting the compound province "Bà Ra - Vng Tàu" a critical Vietnamese geographic edge case where the dash is part of the official name.
3. **Dictionary matching:** Each fragment is matched against `CITY_PATTERNS`. The `normalizeLocation` function additionally filters out false positives by rejecting fragments that match job title keywords (e.g., "chuyên viên", "trng phòng", "giám c") using a Vietnamese occupational regex filter.

6.4.2 Unstructured Salary Categorization (`getSalaryBuckets`)

Salary information poses a significant parsing challenge, often appearing as free text such as "15 - 20 Triu", "Lên n 1000\$", or "Tho thun". The frontend utilizes a `getSalaryBuckets` function executing heuristic regex matching.

The parser first filters out **17 foreign currency codes** (USD, EUR, GBP, JPY, SGD, AUD, CAD, HKD, KRW, THB, MYR, INR, CNY, RMB, TWD, CZK, CHF) via regex to exclude non-VND salaries. It then identifies numeric ranges, standardizes values into millions of VND, and mathematically evaluates the bounds (`[lo, hi]`) against 6 predefined salary tiers. This transforms free text into indexable categorizations like "10 20 triu" without requiring a backend round-trip.

6.4.3 Experience Tier Mapping (`getExpBuckets`)

The `getExpBuckets` parser processes strings like "Di 1 nm", "trên 5 nm", or "2-5 tháng". It handles four Vietnamese expression patterns:

1. **"di" (under):** e.g., "di 1 nm" range `[0, 0.99]`
2. **"trên/hn/ít nht" (over/more than/at least):** e.g., "trên 5 nm" range `[5.01, ]`
3. **Range expressions:** e.g., "1 - 2 nm" range `[1, 2]`

4. **Single values:** e.g., "3 nm" range [3, 3]

It converts all parsed numeric values into a standardized yearly metric (dividing month-based values by 12) and determines intersection overlaps with 4 predefined experience tiers ("Di 1 nm", "1 2 nm", "2 5 nm", "Trên 5 nm"). By executing these deterministic parsers directly in the browser's JavaScript engine, the frontend remains highly responsive while ensuring data conforms strictly to the schema expected by the Elasticsearch backend.

6.5 Page Architecture

| Route | Server/Client | Key Features | |——|:—:|——| | / (Home) | Server + Client | Hero section, search bar with dropdown filters, feature cards | | /search | Client | ES-powered faceted search with 7 filters, pagination, URL-driven state | | /profile | Server + Client | Profile CRUD (experience, education, skills, CV upload via Supabase Storage) | | /insights | Server + Client | Market analytics dashboard | | /login | Client | Login form with Zod validation, Server Action submission | | /signup | Client | Registration form with email confirmation flow | | /job/[id] | Server | Individual job detail page (dynamic route) |

6.6 Key Quantitative Metrics

| Metric | Value | |——|——| | Total page routes | 7 distinct pages | | API routes | 9 route handlers | | Geographic entities in dictionary | 65 (63 provinces/regions + "Toàn quc" + "Nc ngoài") | | Foreign currency codes filtered | 17 | | Salary buckets | 6 ranges | | Experience buckets | 4 ranges | | Work type categories | 5 standard tags | | Vietnamese expression patterns (exp) | 4 distinct regex patterns | | Shared components | 2 (Navbar.tsx, RequireLogin.tsx) | | helpers.ts shared parser | 152 lines |